

eQMS - HLD

1. Purpose & Scope

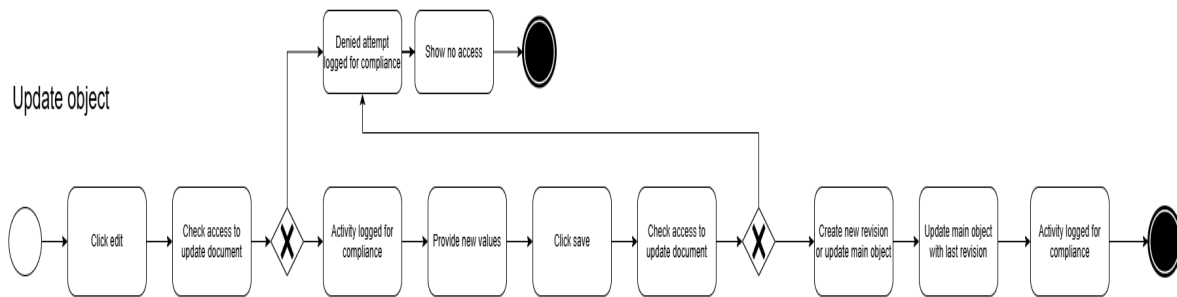
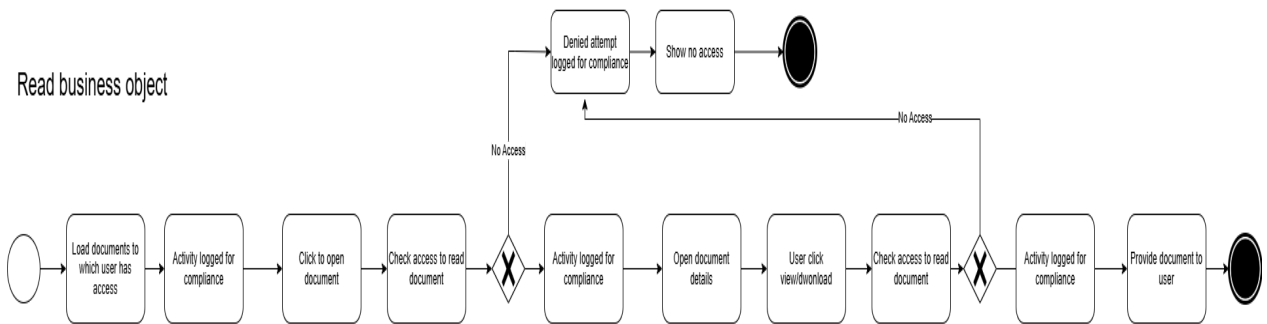
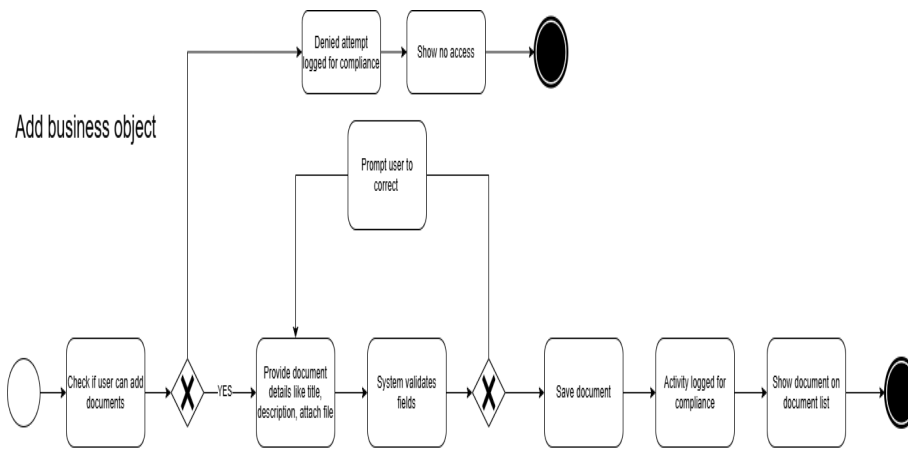
This HLD describes a high-level architecture for a multi-tenant, GxP-capable document management backend with robust audit-trail handling. It implements two audit paths:

- **Transactional audit (CUD operations)** — written transactionally with business changes using an **Outbox** in PostgreSQL, then forwarded to queue and persisted immutably.
- **Access-only audit (Reads/Views)** — produced asynchronously to a durable queue and persisted immutably.

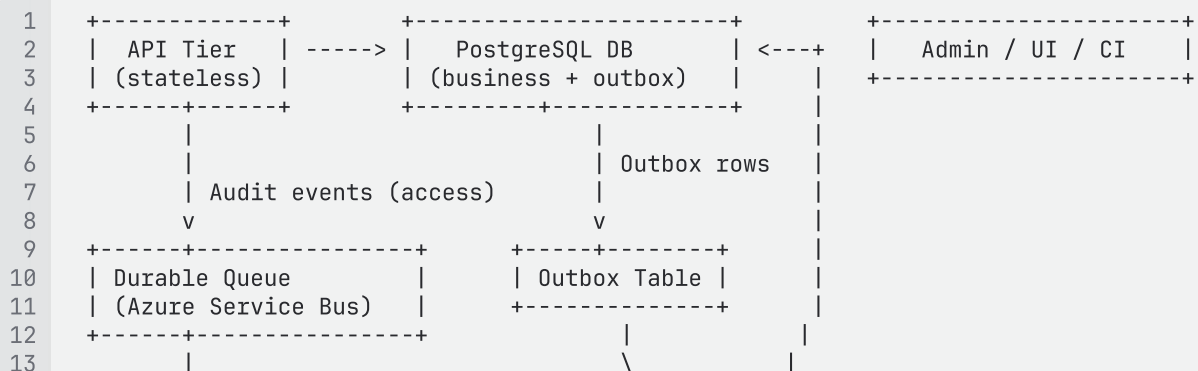
Components use PostgreSQL, Azure Service Bus (recommended) or Azure Table Storage for lightweight queueing/metadata, and Azure Blob Storage (Block/Append blobs with immutability policies) for immutable final audit storage. The Worker component handles queue processing, batching, writing to Blob storage, and outbox draining.

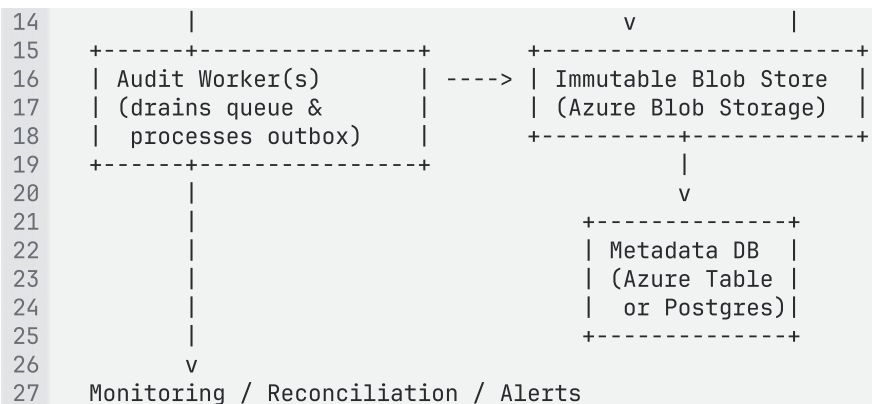
Process Flow

Add document



Architecture Overview (textual)





3. Primary Components

3.1 API Tier (Stateless)

- Exposes REST APIs for document CRUD, listing, and retrieval.
- Performs RBAC and ABAC checks.
- For **CUD** actions: writes business data and an audit payload row to the **Outbox** table in the same DB transaction (Outbox pattern).
- For **Read/View** actions: emits a lightweight audit event message to the **Durable Queue** (non-blocking), or writes to a high-throughput, low-index outbox partition if queue not used.

3.2 PostgreSQL (Single logical database)

- **Business tables** (documents, versions, users, roles, metadata).
- **Outbox table**: same DB, same transaction with business change.
 - Ensures an audit event is persisted only if the business transaction commits.
- Optionally dedicated schema for audit-related transient tables (outbox, dead-letter archive).

3.3 Durable Queue

- **Primary choice: Azure Service Bus (Premium/Standard)** — persistent, duplicate detection, sessions/partitions for tenant ordering.
- **Alternative: Azure Table Storage** used as a simple queue or metadata store for low throughput.
- Queue stores: access-audit events, outbox-forwarded events (if using an outbox-forwarder).

3.4 Audit Worker(s)

- Background service(s) that:
 - Poll/subscribe to Durable Queue.
 - Poll PostgreSQL Outbox for unprocessed entries (if using outbox-forward).
 - Batch events by tenant and time window.

- Write batch files (JSON-lines) to Azure Blob Storage (Block Blob recommended for batching).
- Optionally set immutability/WORM policy or legal hold on the blob after upload.
- Insert a metadata record (index) into Azure Table Storage or the Metadata DB to make events discoverable.
- Mark outbox entries processed (processed_at, audit_blob_ref) or complete queue messages.
- Supports retries, poison-message handling, dead-lettering, alerting.

3.5 Immutable Audit Store

- **Azure Blob Storage (Block Blobs)** configured with immutability policy (WORM).
- Blob naming convention partitions by tenant/time:
`/audit/{tenantId}/{yyyy}/{MM}/{dd}/{HH}/{tenantId}_{yyyyMMddTHHmss}_{batchId}.jsonl`
- Each blob contains a batch of JSON-line audit events; each event includes an `eventId` (UUID) for idempotency.

3.6 Metadata Index (searchable)

- Lightweight index in **Azure Table Storage** or a Postgres metadata table:
 - Contains entries: `tenantId`, `eventId`, `timestamp`, `actionType`, `objectType`, `objectId`, `blobUri`, `offset` (if needed), `policyVersion`.
- Used to support quick filtering queries and to assemble exports (CSV) by auditor requests.

3.7 Monitoring & Reconciliation

- Metrics, alerts, and scheduled reconciliation jobs:
 - Outbox backlog size, unprocessed queue length, worker failure counts.
 - Reconciliation compares outbox/queue → metadata → blob contents to ensure no missing events.

3.8 Compliance Access Layer

- A read-only, audited UI/API for Compliance/QA to query metadata and request audit exports.
- Only dedicated compliance roles can access.

4. Data Flows & Sequences

4.1 Transactional Audit (CUD) — Sequence

1. Client calls API to create/update/delete a Document.
2. API performs business logic and opens DB transaction.
3. Within same transaction:
 - Apply business changes.
 - Insert an Outbox row with an audit payload `{eventId, tenantId, action, objectType, objectId, actor, timestamp, policyIds, payload}`.
4. Commit transaction → Outbox row now durable.
5. Audit Worker polls Outbox (or Outbox Forwarder pushes to queue):
 - Reads unprocessed Outbox rows (batch).
 - Persists batch to Blob storage (a single JSONL file).
 - Inserts metadata index records for each event (with blobUri + offset).
 - Marks Outbox rows processed and saves `processed_at` , `blobRef` .
6. Audit Worker deletes or archives processed outbox rows after a retention window (retention & reconciliation kept).

Notes: The outbox entry is only present if business commit succeeded — no audit for rolled-back business operations.

4.2 Access-only Audit (READ) — Sequence

1. Client calls API to GET document metadata or content.
2. API performs ABAC checks. If allowed:
 - Returns document content (or streams).
 - **Asynchronously** (non-blocking) the API publishes a lightweight audit event to the Queue:
`{eventId, tenantId, action: "READ", objectType, objectId, actor, timestamp, fieldsReturned (list)}`
3. Audit Worker(s) consume queue messages, batch by tenant/time, write to blob, update metadata index.
4. Queue acknowledges the message only after success (PeekLock/Complete semantics).

Notes: API does not block for audit persistence; queue and worker guarantees durability and at-least-once delivery.

5. Key Data Schemas

5.1 Outbox table (Postgres)

```

1 CREATE TABLE outbox (
2   id BIGSERIAL PRIMARY KEY,
3   event_id UUID NOT NULL,
4   tenant_id UUID NOT NULL,
5   aggregate_type TEXT NOT NULL,
6   aggregate_id TEXT,
7   event_type TEXT NOT NULL,
8   payload JSONB NOT NULL,
9   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
10  processed_at TIMESTAMPTZ NULL,
11  processing_attempts INT NOT NULL DEFAULT 0
12 );
13
14 CREATE INDEX idx_outbox_unprocessed ON outbox (created_at) WHERE processed_at IS NULL;
15

```

5.2 Metadata Index (Azure Table / Postgres)

If Azure Table:

- PartitionKey = tenantId
- RowKey = {timestampTicks}-{eventId}
- Columns: eventType, objectType, objectId, blobUri, payloadSummary, policyVersion

If Postgres:

```

1 CREATE TABLE audit_index (
2   event_id UUID PRIMARY KEY,
3   tenant_id UUID NOT NULL,
4   event_time TIMESTAMPTZ NOT NULL,
5   action TEXT NOT NULL,
6   object_type TEXT,
7   object_id TEXT,
8   blob_uri TEXT NOT NULL,
9   blob_offset INT,
10  payload_summary JSONB,
11  policy_version TEXT
12 );
13 CREATE INDEX idx_audit_tenant_time ON audit_index (tenant_id, event_time);
14

```

5.3 Blob payload (JSON Lines)

Each line is a JSON object:

```

1 {
2   "eventId": "uuid",
3   "tenantId": "uuid",
4   "timestamp": "2025-11-15T12:34:56Z",
5   "actor": {"userId": "u-..", "roles": ["Reader"]},
6   "action": "Document.View",
7   "objectType": "Document",
8   "objectId": "doc-123",
9   "fieldsReturned": ["title", "status"],
10  "policyIds": ["p-1", "p-2"],
11  "sourceIp": "1.2.3.4",
12  "notes": null
13 }
14

```

eliability, Ordering & Idempotency

• Durability guarantees

- Outbox rows are stored transactionally with business changes.
- Queue messages persisted (Service Bus durability).
- Once Audit Worker writes to blob and updates index, event considered persisted.

• Ordering

- Per-tenant ordering preserved by batching in Worker (prefer Session/Partition keys in Service Bus).

• Idempotency

- Each event includes `eventId`. Audit Worker must be idempotent (check if `eventId` already present in metadata).

• Retries

- Worker retries on failures; after N attempts move to Dead-Letter/Archive with alert.

7. Scaling & Performance

- **API tier:** Stateless, horizontal scale (Kubernetes/App Service), behind load balancer.
- **Postgres:** scale vertically initially; use read replicas for heavy reads. Partition large `audit_index` if needed.
- **Outbox:** minor overhead; indexes restricted to unprocessed rows for speed.
- **Queue:** Service Bus scales; use partitions/sessions for concurrency (+ throughput).
- **Audit Worker(s):** autoscale by queue length; workers can be sharded by tenant or time window.
- **Blob Storage:** unlimited scale; use per-tenant prefixing to reduce hotspotting.
- **Batching:** tune batch window (e.g., 1–60s) and batch size to trade latency vs throughput and transaction frequency.

8. Security & Compliance Controls

• Authentication & Authorization

- Central identity provider (OIDC/SAML) for users and admin roles.
- API tokens with tenant-scoped claims.

• Data protection

- TLS in transit.
- Encryption at rest: Postgres (TDE), Blob Storage server-side encryption (Customer-managed keys optional).

- **Immutability**

- Set Blob immutability policies (WORM/legal hold) on uploaded audit blobs immediately after upload.
- Audit Worker sets immutability and stores checksum/hash of each blob.

- **Access control**

- App and Worker have `insert` /write rights to metadata and blob, but only Compliance role has read across tenants.
- Database-level role separation: schema for app vs compliance.

- **Change control & validation**

- All policy and code changes go through configured change management (SOPs).

- **Tamper evidence**

- Optional chain-hash per audit entry stored in metadata for verification.

9. Monitoring, Alerts & Reconciliation

- **Metrics**

- Outbox unprocessed count, average processing time.
- Queue depth and message age.
- Worker errors, DLQ counts.
- Blob upload success/failure rates.

- **Alerts**

- Outbox backlog > threshold
- Queue messages > threshold
- Reconciliation mismatch found
- Worker repeated failures or DLQ entries

- **Reconciliation jobs**

- Nightly job to compare `outbox` processed rows and `audit_index` /blob contents.
- Report mismatches and create remediation tasks.

- **Auditability**

- All reconciliation runs are themselves logged to immutable audit store.

10. Backup, Retention & Recovery

- **Outbox retention:** keep processed rows for N days (7–30) for re-delivery diagnostics; archived before purge.
- **Blob retention:** policy per tenant (e.g., 7+ years), using immutability policies.

- **Metadata index retention:** keep for search convenience; consider moving older index to cold storage.
- **Disaster recovery:**
 - Postgres backups (scheduled, point-in-time recovery).
 - Blob replication for geo resilience (RA-GRS if needed).
 - Queue redundancy via Service Bus Geo-DR features if needed.
- **Restore**
 - Procedure to rehydrate audit index from blobs if index lost.

11. Operational Runbook (high-level)

Normal flow

- API writes outbox or queue → Worker persists to blob → metadata updated → outbox cleaned.

Failure scenarios

- Worker fails: monitor alerts; process DLQ and archived outbox; restart workers.
- Blob upload fails: worker retries with backoff; on repeated failure move to dead-letter archive and alert ops.
- Reconciliation mismatch: generate report; investigate outbox/worker logs; reprocess batches as needed.

Manual reprocess

- Reprocess unprocessed outbox rows or DLQ events to recreate blobs or metadata entries.

12. Security & Compliance Checklist (for GxP)

- Unique user IDs; strong authentication; MFA for admin roles.
- RBAC + ABAC enforced at API layer.
- Immutable audit blobs with WORM policy enabled.
- Audit metadata indexed and exportable; Compliance-only read access.
- All cryptographic keys in a secrets service (Key Vault).
- SOPs for change control, backup/restore, retention, reconciliation.
- IQ/OQ/PQ validation plan for the whole pipeline.